[Week 16]

chelseey · 2025년 6월 12일

통계 수정 삭제

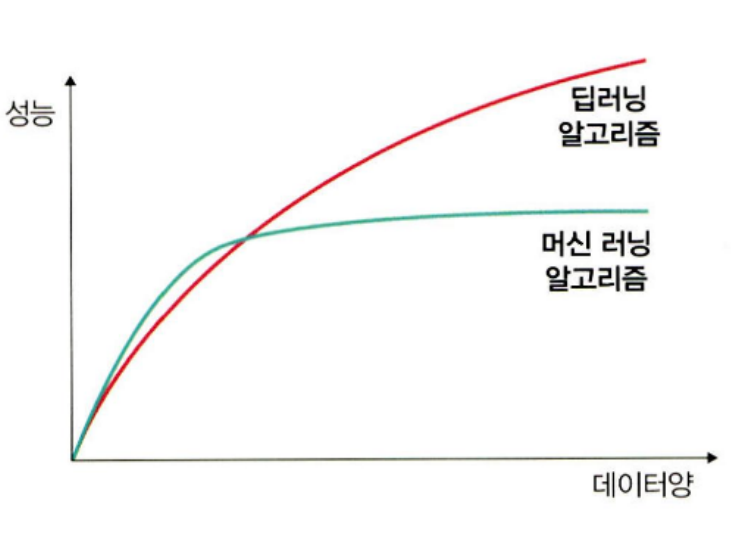
Euron

8.1 성능 최적화

데이터를 사용한 성능 최적화

가능한 한 많은 데이터를 확보·활용하여 모델 성능을 높이는 방법

최대한 많은 데이터 수집하기



일반적인 머신러닝·딥러닝 알고리즘은 데이터량이 많을수록 성능이 개선됨

데이터 생성(Data Augmentation)

충분한 실제 데이터를 수집하기 어려울 때는 인위적으로 데이터를 만들어 사용

데이터 범위(Scale) 조정

활성화 함수별로 입력값 범위를 맞춰주면 학습 안정성이 향상됨

- Sigmoid 사용 시 → 데이터 범위를 0~1로
- Tanh 사용 시 → 데이터 범위를 –1~1로

알고리즘을 이용한 성능 최적화

1. 다양한 후보 알고리즘 선정
   같은 문제(분류·회귀·시계열 등)에 적용할 수 있는 유사한 용도의
   알고리즘을 여러 개 준비
2. 모델 훈련 및 검증
   각 알고리즘으로 동일한 데이터(또는 동일한 교차검증 절차)로 학습,
   평가 지표로 비교
3. 최적 알고리즘 선택

알고리즘 튜닝을 위한 성능 최적화

하나의 모델을 선택한 뒤, 각종 하이퍼파라미터를 바꿔가며
학습·검증을 반복하여 최적의 성능을 찾는 과정

주요 하이퍼파라미터

- 진단(Diagnosis)
  학습(train) vs. 검증(test) 곡선을 비교
  학습이 검증보다 월등히 좋으면 과적합 의심 → 규제
  양쪽 모두 낮으면 과소적합 의심 → 모델 복잡도↑ 또는 에포크↑
  검증이 학습을 넘어가면 조기 종료(early stopping) 고려
- 가중치 초기화
  작은 난수로 시작
  필요시 오토인코더 같은 사전 학습(pre-training)으로 초기 가중치 확보
- 학습률(Learning Rate)
  네트워크 깊이에 따라 다르게 설정
  계층 많으면 큰 값, 얕으면 작은 값으로 시작해 조정
- 활성화 함수
  변경 시 손실 함수(loss)와 함께 조합
  은닉층은 ReLU 계열, 출력층은 문제 특성에 맞춰 Softmax/Sigmoid 선택
- 배치 크기 & 에포크
  최근 트렌드: 작은 배치, 많은 에포크
  데이터 크기와 GPU 메모리 상황 고려해 다양한 조합 실험
- 옵티마이저 & 손실 함수
  Adam, RMSProp 등 확률적 경사하강법 계열 사용이 일반적
- 네트워크 구성(Topology)
  층 수, 각 층의 유닛 수, 잔차 연결(residual) 여부 등 구조 변경
  복잡도를 높이면 표현력↑, 과적합 위험↑

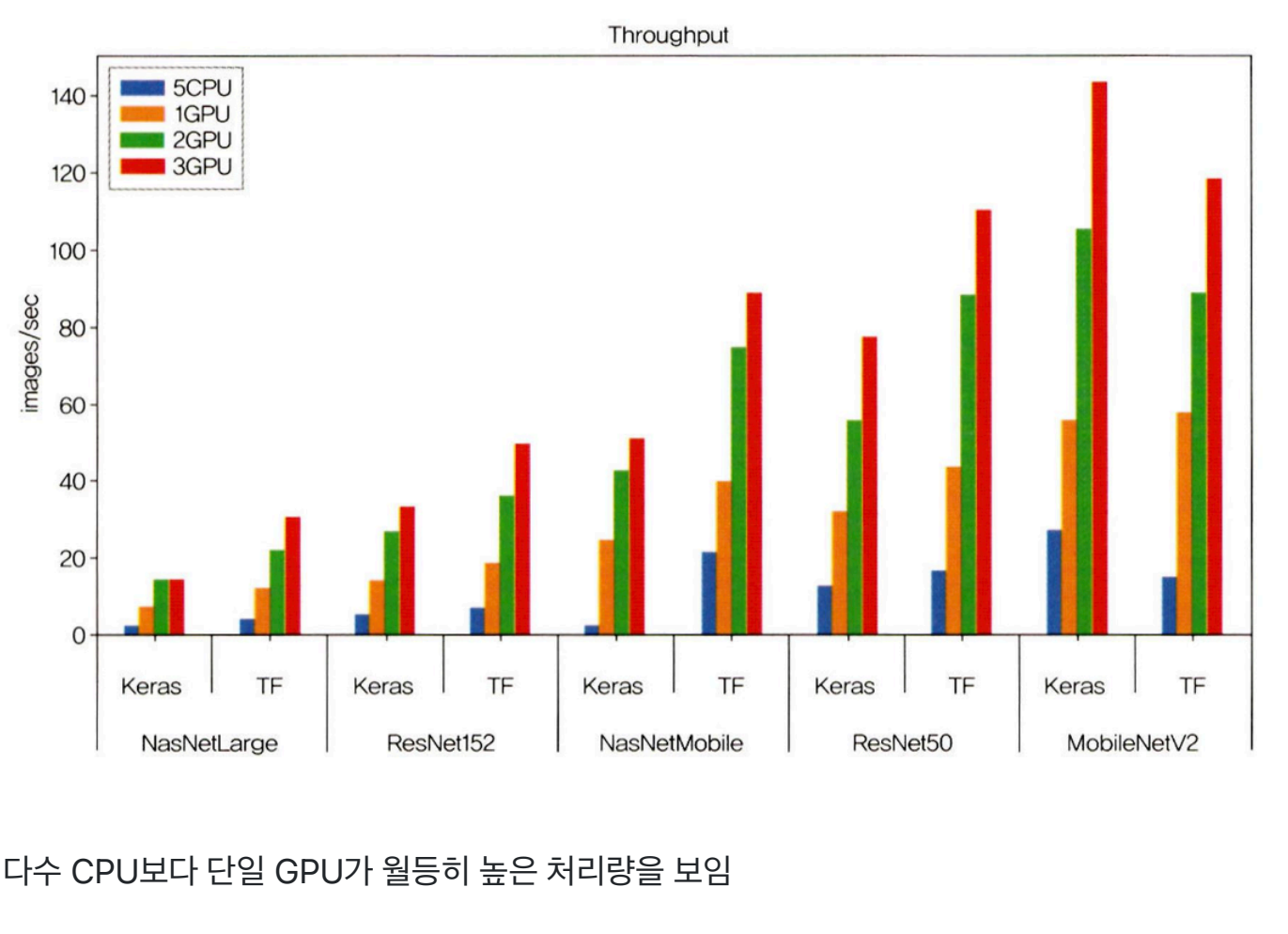
앙상블을 이용한 성능 최적화

앙상블(Ensemble)
: 두 개 이상의 서로 다른(혹은 같은) 모델을 결합해 예측을 수행

여러 모델의 강점을 모아 단일 모델의 한계를 극복하고,
예측 안정성과 정확도를 높임.

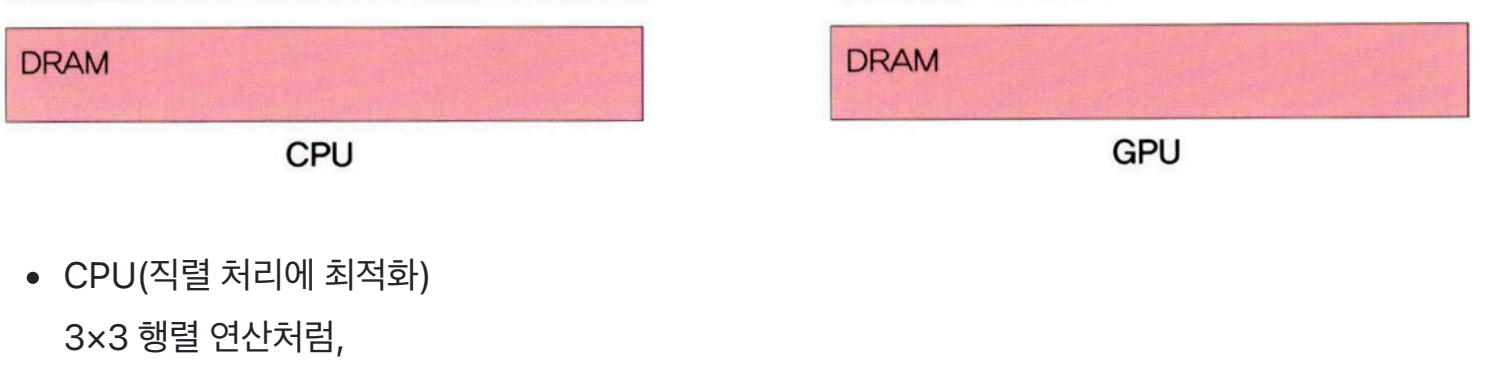
8.2 하드웨어를 이용한 성능 최적화

CPU 와 GPU 사용의 차이



다수 CPU보다 단일 GPU가 월등히 높은 처리량을 보임

구분	CPU	GPU
목적	범용 명령어 처리(Control + Cache + ALU 소수)	대량 병렬 연산(대수 연산용 ALU 다수)
연산 방식	순차적: 한 번에 하나의 명령어 실행	병렬적: 다수의 명령을 동시에 처리
ALU 개수	적음(수십 개 이하)	많음(수백~수천 개)
캐시 메모리	큼(명령어 해석·제어 중심)	작음(연산 위주, 메모리 병목 낮춤)
최적 분야	복잡한 분기·제어 흐름이 필요한 작업	대규모 선형 대수 연산(행렬·벡터 연산), 3D 그래픽 렌더링 등 병렬 작업



- CPU(직렬 처리에 최적화)
  3×3 행렬 연산처럼,
  A·B·C 열을 순서대로 처리하는 재귀적(직렬) 연산에 적합
  한 번에 하나의 명령만 실행 → 복잡한 제어 흐름·분기 처리에 강함
- GPU(병렬 처리에 최적화)
  backpropagation처럼 동일한 연산을 수백만 차례 반복해야 할 때 유리
  수백~수천 개의 ALU를 병렬로 사용해 여러 명령을 동시에 처리

GPU 를 이용한 성능 최적화

CUDA(CUDA Toolkit)와 cuDNN 사용

CUDA : NVIDIA에서 개발한 GPU 병렬 연산 프레임워크

8.3 하이퍼파라미터를 이용한 성능 최적화

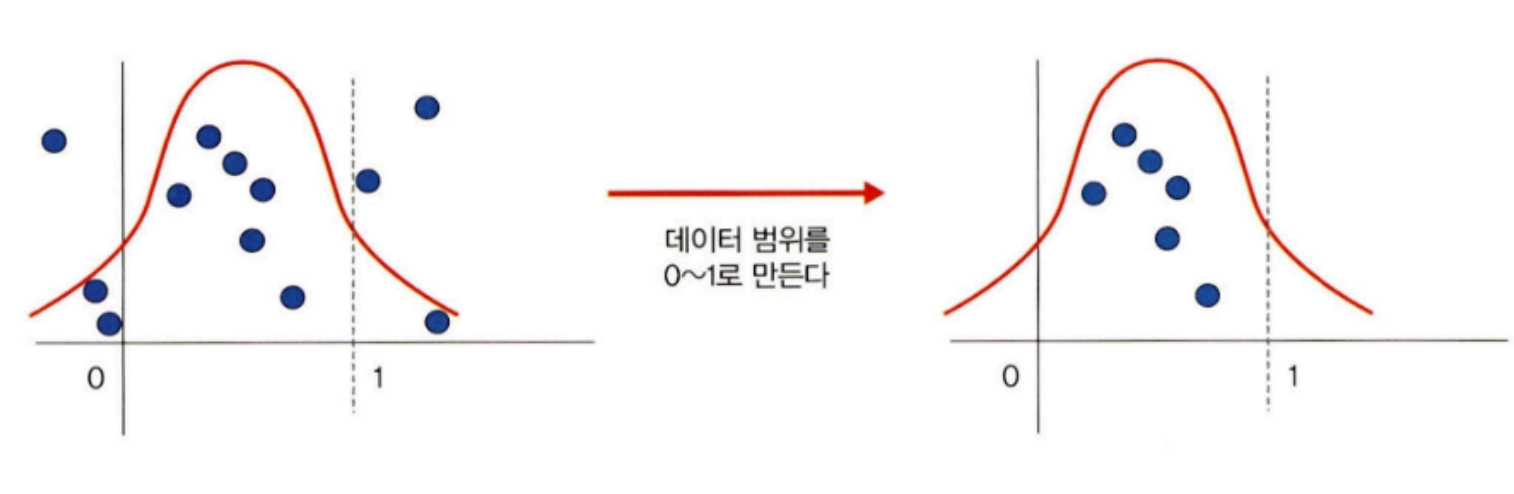
배치 정규화 , 드롭아웃 , 조기 종료

배치 정규화를 이용한 성능 최적화

정규화(Normalization)

각 특징(feature)의 값을 사용자가 원하는 범위로 제한
ex. 이미지 픽셀 값(0~255)을 0~1 구간으로 변환

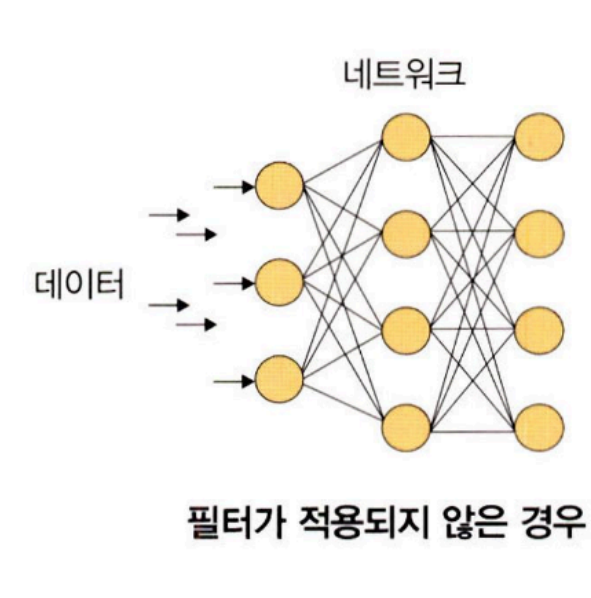
Min–Max Scaling



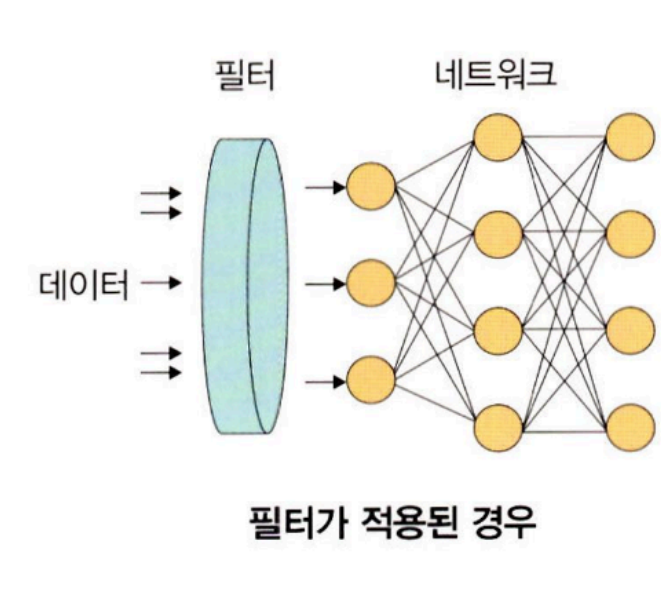
$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

규제화 (Regularization)

모델 복잡도를 줄여 과적합(overfitting) 방지
네트워크에 필터를 씌워 불필요한 노드(파라미터)가 지나가지 못하도록 제약



필터가 적용되지 않은 경우



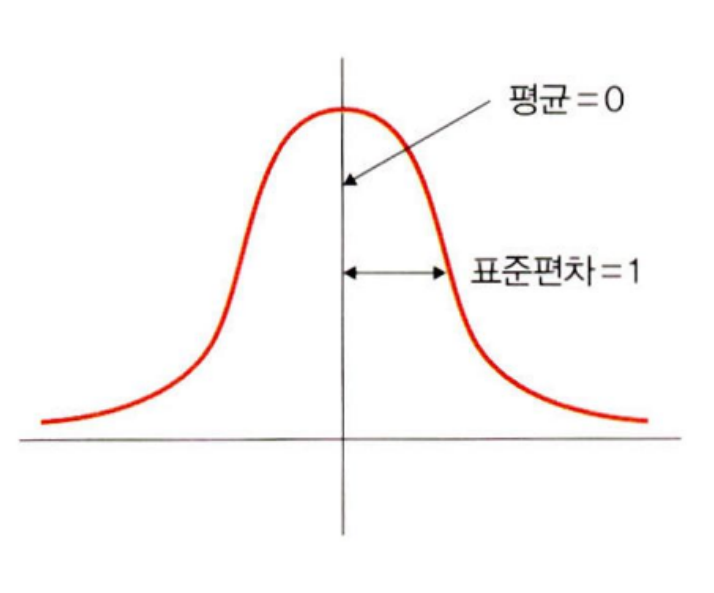
필터가 적용된 경우

- 드롭아웃(Dropout)
  학습 시 무작위로 일부 뉴런을 비활성화 → 강건성 향상
- 조기 종료(Early Stopping)
  검증 손실이 더 이상 개선되지 않을 때 학습 중단 → 과적합 예방

표준화(Standardization)

데이터 분포를 평균 0, 분산 1 형태로 변환
가우시안(정규) 분포 가정 하에 모델 학습 안정성 및 수렴 속도 개선

Z-score Normalization

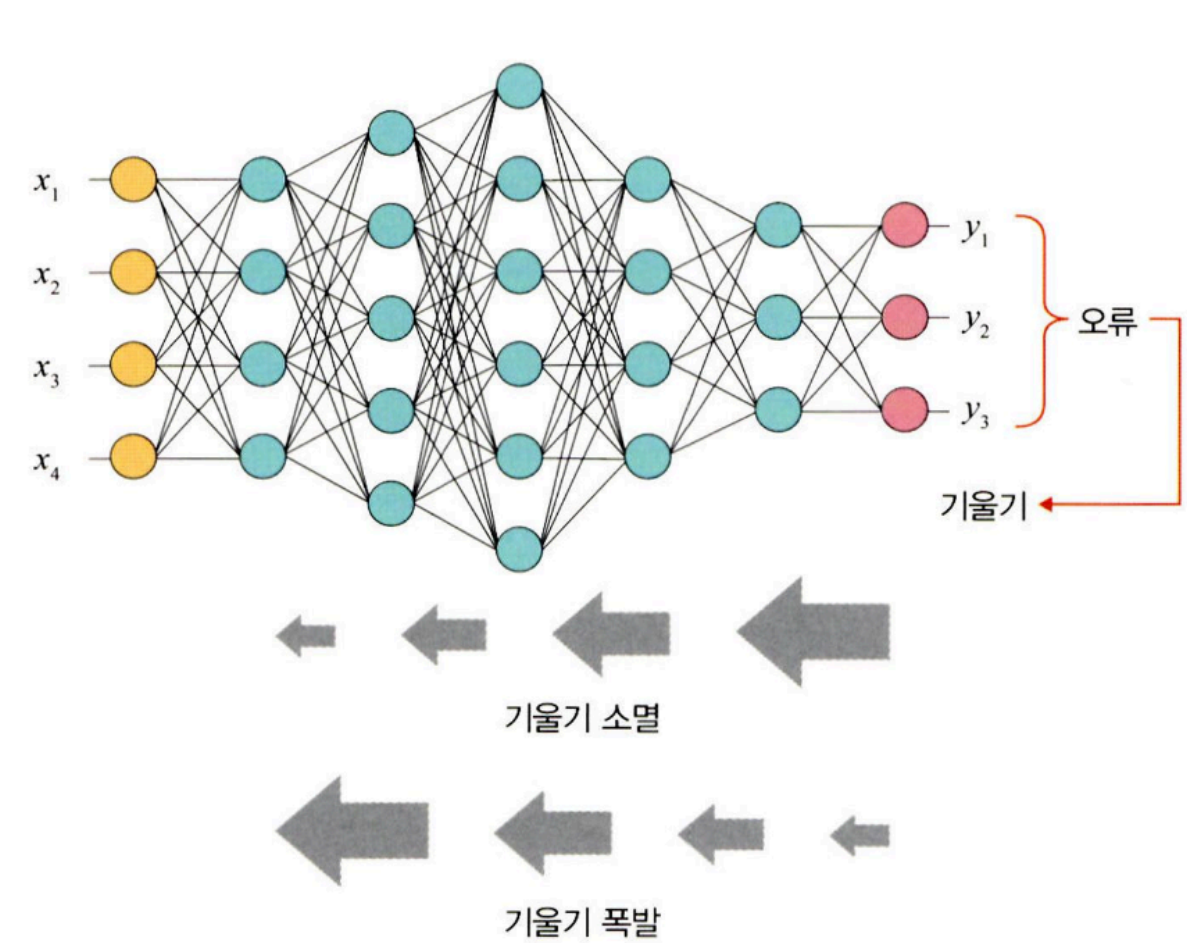


$$x_{\text{std}} = \frac{x - \mu}{\sigma}$$

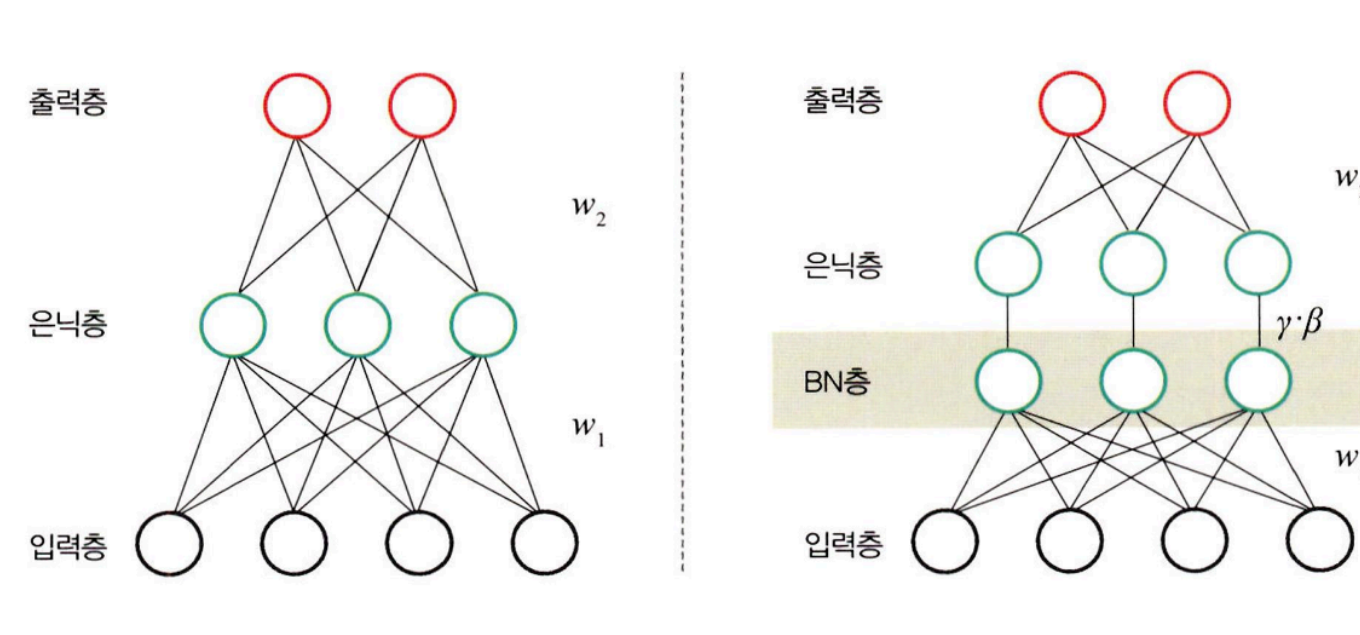
배치 정규화(Batch Normalization)

기울기 소멸(gradient vanishing), 기울기 폭발(gradient exploding) 문제 완화

기울기 소멸과 기울기 폭발



- 기울기 소멸 : 오차 정보를 역전파시키는 과정에서
  기울기가 급격히 0 에 가까워져 학습이 되지 않는 현상
- 기울기 폭발 : 학습 과정에서 기울기가 급격히 커지는 현상



1. 미니배치 평균
   배치 내 모든 샘플의 평균 계산

$$\mu_\beta \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i$$

2. 분산 및 표준편차
   배치 내 분산
   작은 상수(ε)로 분모 안정화

$$\sigma_\beta^2 \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_\beta)^2$$

3. 정규화(Normalization)
   평균 0, 분산 1

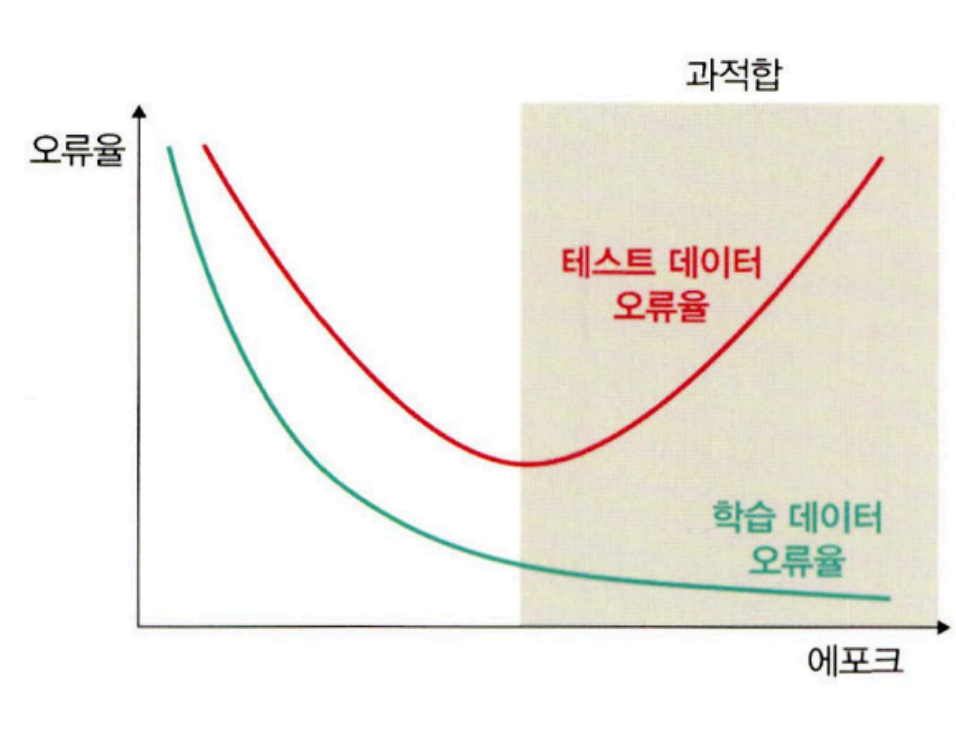
$$\hat{x}_i \leftarrow \frac{x_i - \mu_\beta}{\sqrt{\sigma_\beta^2 + \epsilon}}$$

4. 스케일 및 시프트
   학습 가능한 하이퍼파라미터 γ,β 적용

$$y_i \leftarrow \gamma \hat{x}_i + \beta \quad (\equiv \mathrm{BN}_{\gamma,\beta}(x_i))$$

→ 네트워크 층(layer)마다 입력 분포를 고정시켜 학습 안정성 및 수렴 속도 향상

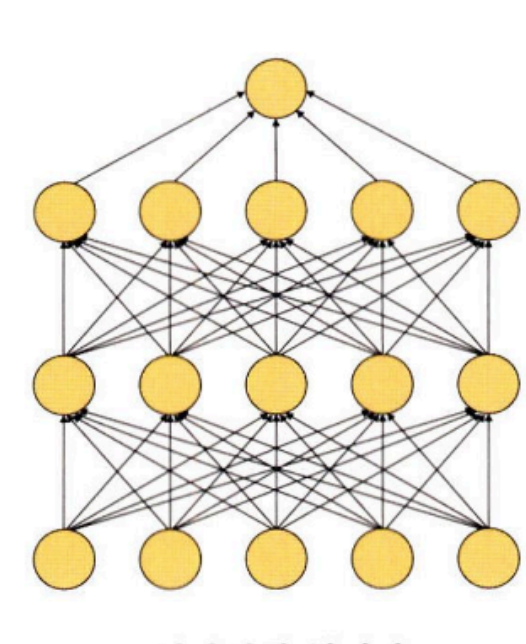
드롭아웃을 이용한 성능 최적화



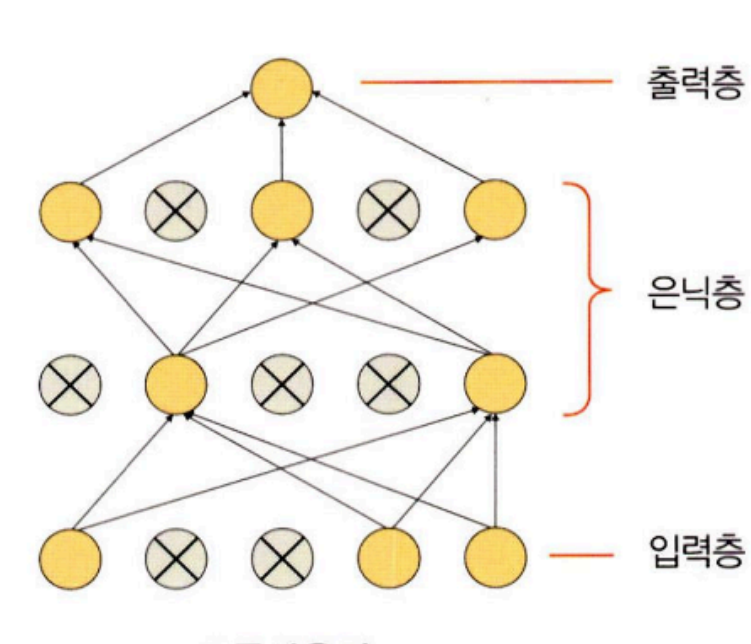
과적합(Over-fitting): 훈련 데이터에 너무 치우쳐 학습하면

- 학습 데이터 오류는 계속 줄지만
- 테스트 데이터 오류는 일정 시점부터 다시 증가 → 일반화 성능 저하

드롭아웃



일반적인 신경망



드롭아웃이 적용된 신경망

훈련 시 : 은닉층의 뉴런을 임의로 off

- 각 배치마다 랜덤으로 일정 비율(p)을 선택해 비활성화
- 꺼진 뉴런은 순전파·역전파 시 신호를 전달하지 않음

테스트 시 : 모든 뉴런 사용

- 훈련 때 비활성화 비율에 맞춰 출력값을 스케일링 (또는 가중치에 곱해 보정)
- ex. p=0.5라면 활성 뉴런 출력만 2배

배치에서 이미지-레이블 분리

```
dataiter = iter(trainloader)
images, labels = dataiter.next()

print(images.shape)    # images.shape = (배치 크기, 채널, 높이, 너비)
print(images[0].shape)
print(labels[0].item())
```

배치 정규화 미적용 모델

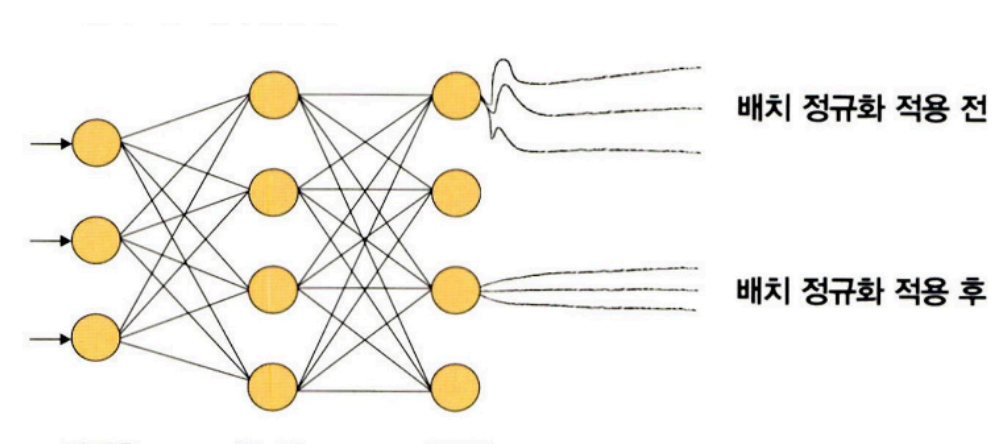
```
class NormalNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.classifier = nn.Sequential(
            nn.Linear(784, 48),
            nn.ReLU(),
            nn.Linear(48, 24),
            nn.ReLU(),
            nn.Linear(24, 10)
        )
    def forward(self, x):
        x = x.view(x.size(0), -1)      # (B,1,28,28) → (B,784)
        return self.classifier(x)
```

- Fully-connected 레이어와 ReLU만 사용
- 은닉층 전후 입력 분포 변화에 대한 보정 없음

배치 정규화 적용 모델

```
class BNNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.classifier = nn.Sequential(
            nn.Linear(784, 48),
            nn.BatchNorm1d(48),      # ← 배치 정규화
            nn.ReLU(),
            nn.Linear(48, 24),
            nn.BatchNorm1d(24),      # ← 배치 정규화
            nn.ReLU(),
            nn.Linear(24, 10)
        )
    def forward(self, x):
        x = x.view(x.size(0), -1)
        return self.classifier(x)
```

- 각 은닉층 직후에 BatchNorm1d 삽입
- 배치 정규화는 내부 공변량 변화를 억제해 학습을 안정화



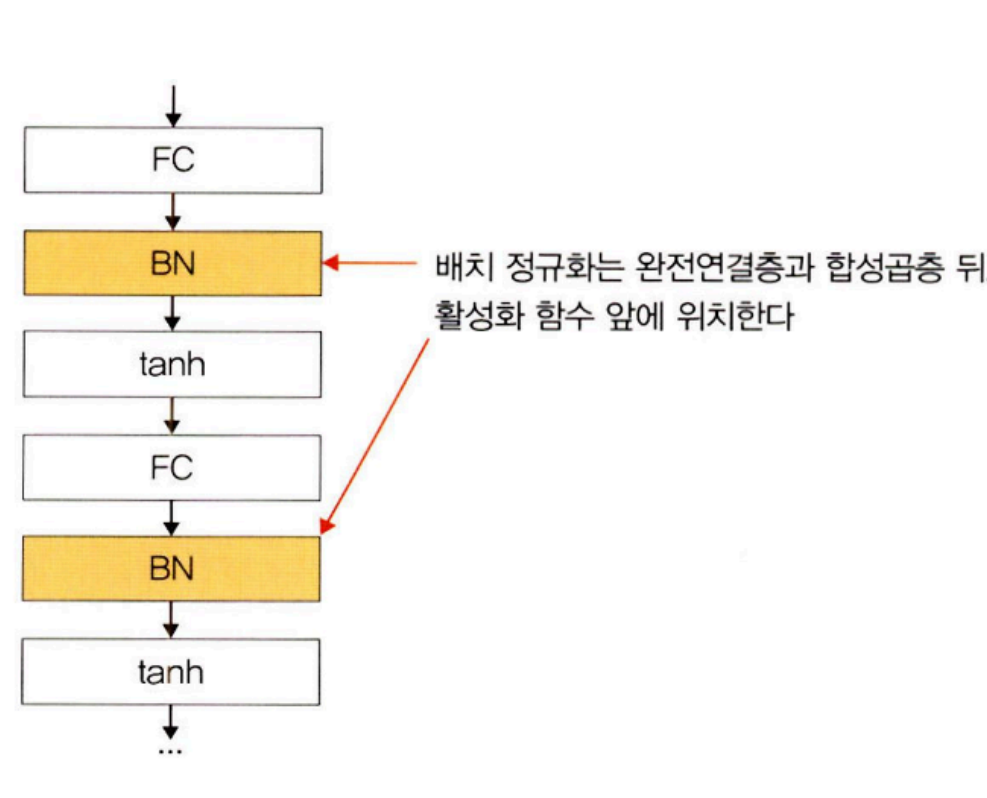

FC
BN
tanh
FC
BN
tanh
…
배치 정규화는 완전연결층과 합성곱층 뒤, 활성화 함수 앞에 위치한다

그림 8-41 첫 번째 에포크에 대한 학습 결과

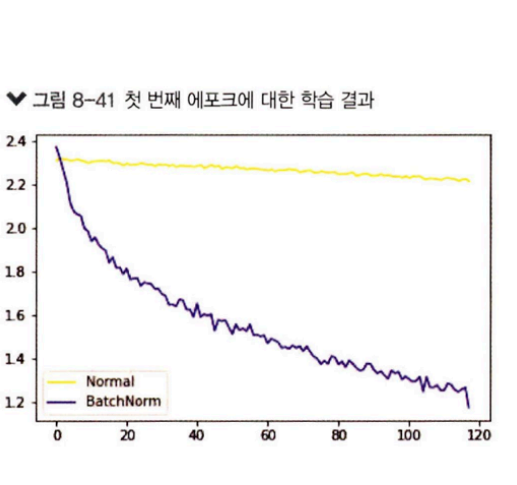
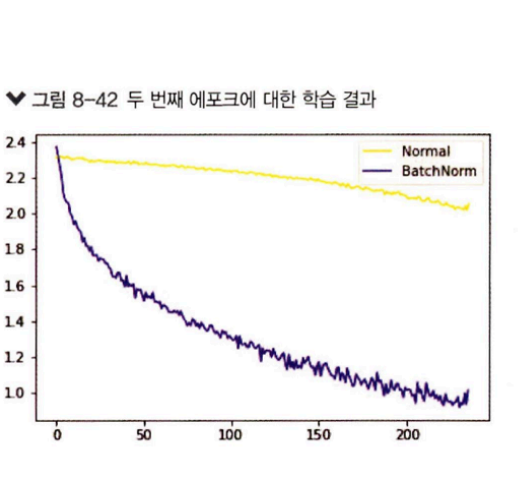


그림 8-42 두 번째 에포크에 대한 학습 결과



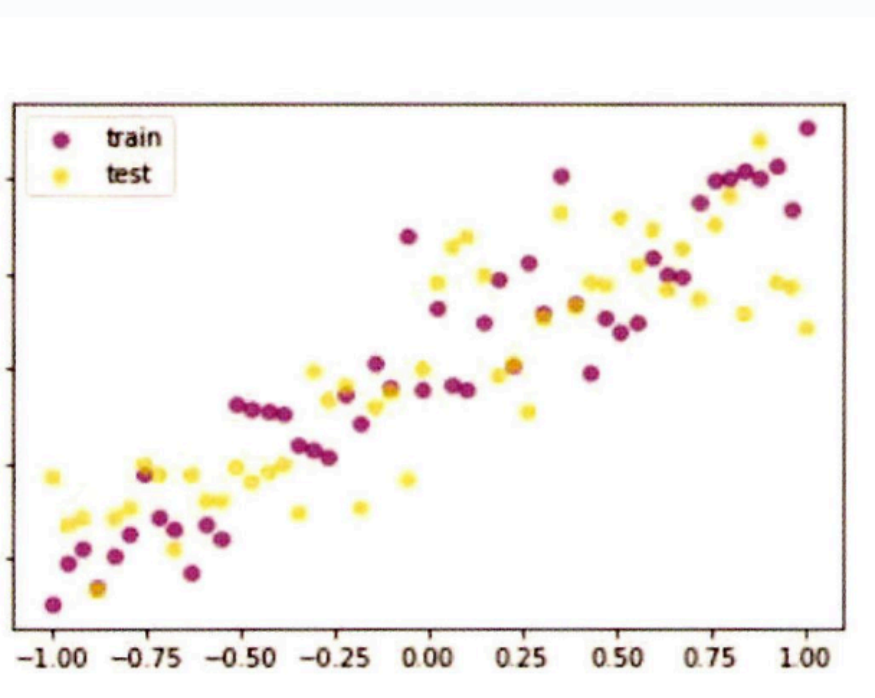
훈련-테스트용 1D 회귀 데이터 생성

```
N      = 50
noise = 0.3

# - x_train: -1…1 구간에서 균등하게 N개 생성 → (N,1)
x_train = torch.unsqueeze(torch.linspace(-1, 1, N), 1)
# - y_train: x_train에 평균 0, σ=1 정규분포 노이즈 추가
y_train = x_train + noise * torch.normal(torch.zeros(N,1), torch.ones(N,1))

# x_test, y_test: 같은 방식으로 테스트용 데이터 생성
```

- `linspace` 로 구간 생성, `unsqueeze` 로 (N,)→(N,1) 차원 확대
- `torch.normal(zeros, ones)` 로 평균 0·표준편차 1 가우시안 샘플링

데이터 분포 시각화

```
plt.scatter(x_train .data.numpy(), y_train .data.numpy(),
            c='purple', alpha=0.5, label='train')
plt.scatter(x_test  .data.numpy(), y_test  .data.numpy(),
            c='yellow', alpha=0.5, label='test')
plt.legend(); plt.show()
```



훈련과 테스트 데이터가 고르게 분포되어 있음

드롭아웃 적용 vs. 미적용 모델 학습 결과 비교

그림 8–46 첫 번째 드롭아웃에 대한 학습 결과

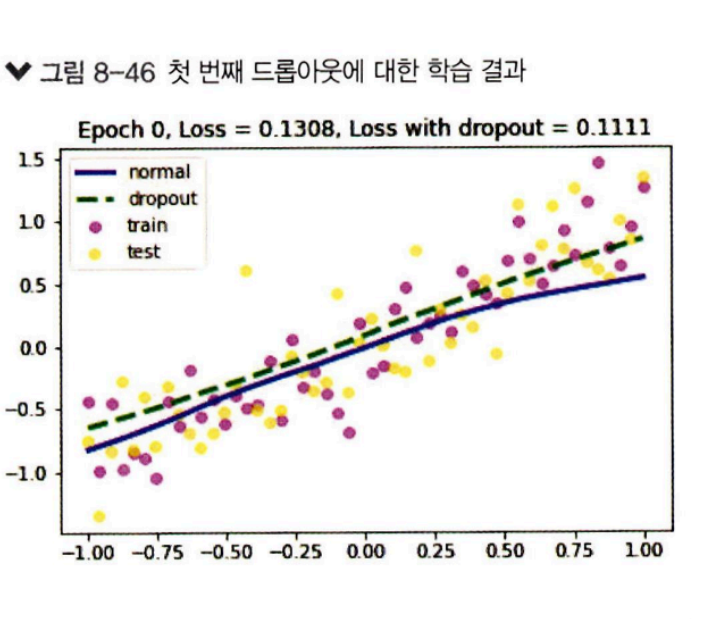
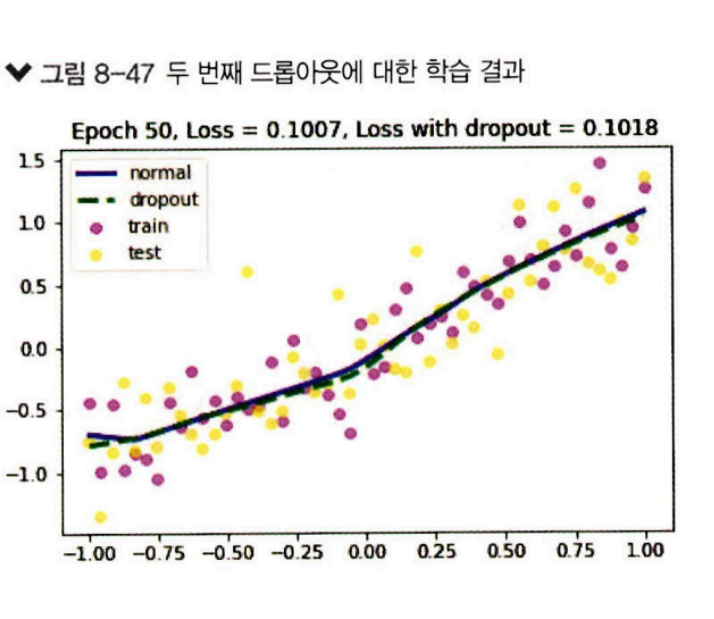


그림 8–47 두 번째 드롭아웃에 대한 학습 결과



… 중간 생략 …

그림 8–48 19번째 드롭아웃에 대한 학습 결과

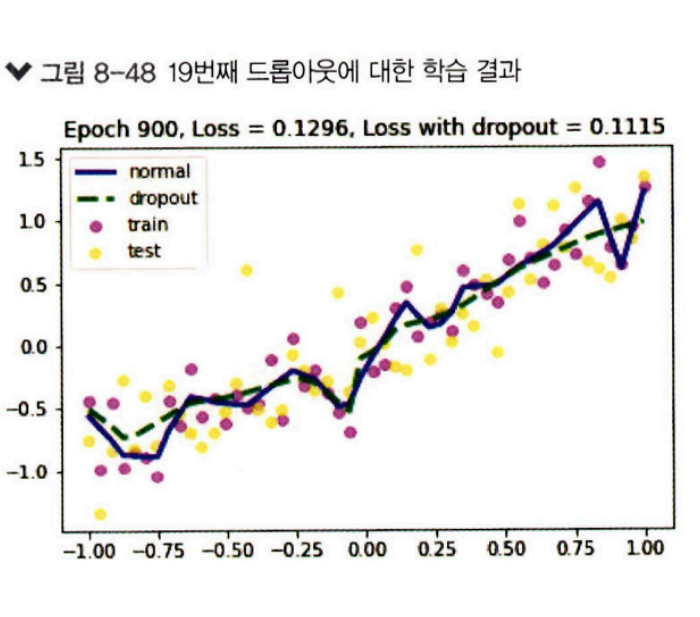
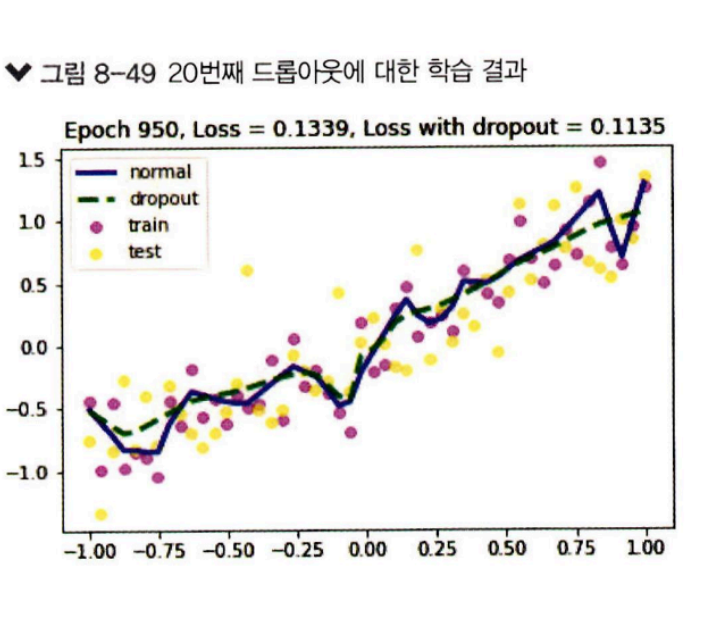


그림 8–49 20번째 드롭아웃에 대한 학습 결과

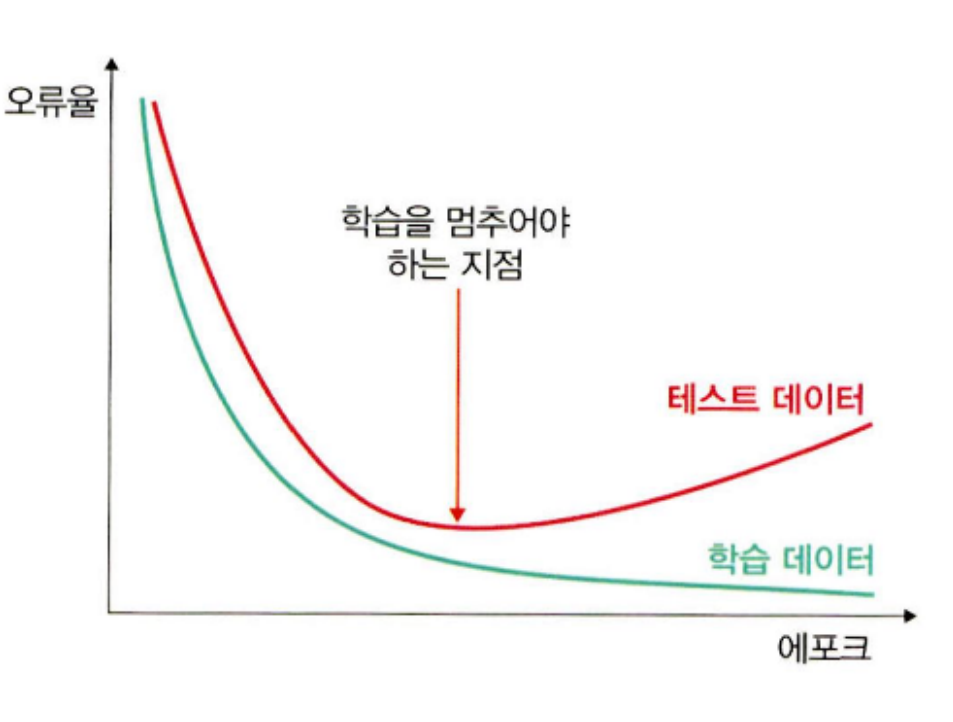


- 초기 수렴 속도
  : 드롭아웃 모델이 다소 느림
- 최종 손실
  : 일반 모델 대비 드롭아웃 모델이 과적합 억제 효과로 더 낮음
- 곡선 형태
  : 드롭아웃 모델이 더욱 부드럽게 수렴, 진동(오버슈팅) 감소

조기 종료를 이용한 성능 최적화

과적합 방지용 규제 기법

- 검증 데이터를 별도로 두고,
  매 에포크마다 검증 손실(validation loss)을 측정
- 검증 손실이 증가하기 시작하는 시점(과적합 발생 직전)에 학습을 중단



훈련 손실(training loss)과 검증 손실(validation loss) 모두 학습 초기에는 감소

일정 시점 이후,

- 훈련 손실은 계속 감소
- 검증 손실은 오히려 증가 → 모델이 훈련 세트에 과적합

이 `검증 손실 최저점` 에서 학습을 멈추면
과적합 이전의 모델 파라미터를 확보할 수 있음

데이터셋 전처리(Transforms) 정의

```
train_transform = transforms.Compose([
    transforms.Resize( (224,224) ),        # 모델 입력 크기 맞춤
    transforms.RandomHorizontalFlip(),     # 좌우 뒤집기(데이터증강)
    transforms.RandomVerticalFlip(),       # 상하 뒤집기(데이터증강)
    transforms.ToTensor(),                 # [0~255]→[0.0~1.0] 텐서
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],        # ImageNet 사전학습 통계
        std= [0.229, 0.224, 0.225]
    )
])

val_transform = transforms.Compose([
    transforms.Resize( (224,224) ),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std= [0.229, 0.224, 0.225]
    )
])
```

`RandomHorizontalFlip` / `RandomVerticalFlip` : 훈련 시 데이터 증강

학습률 스케줄러

```
class LRScheduler():
    def __init__(self, optimizer, patience=5, min_lr=1e-6, factor=0.5):
        self.lr_scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
            optimizer,
            mode='min',         # val_loss 기준 → 떨어지지 않으면 lr 감소
            patience=patience,  # 개선 없을 때 대기 에포크 수
            factor=factor,      # lr ← lr * factor
            min_lr=min_lr,      # lr 하한선
            verbose=True        # lr 변화 로그 출력
        )

    def __call__(self, val_loss):
        # 검증 손실을 입력받아 스케줄러 스텝 실행
        self.lr_scheduler.step(val_loss)
```

`ReduceLROnPlateau`

- mode='min' : 검증 손실이 더 이상 내려가지 않을 때 작동
- patience : 개선이 없을 때 몇 에포크 기다릴지
- factor : lr을 얼마만큼 줄일지
- min_lr : lr 하한선
- verbose : lr 감소 시 콘솔에 메시지 출력

→ 매 에포크 검증 손실(`val_loss`)을 `scheduler(val_loss)` 로 전달

콜백(Callback)

개발자가 직접 호출하지 않고,
학습 루프에서 스케줄러에 `val_loss` 를 넘기면 내부적으로 작동하는 함수

- 동기적(synchronous) 콜백
  코드가 위→아래, 왼쪽→오른쪽으로 순차적으로 실행되는 함수

- 비동기적(asynchronous) 콜백
  병렬 처리와 동일.
  코드가 완료될 때까지 오래 걸리면 기다리지 않고,
  다른 코드를 먼저 처리되도록 함.

조기 종료 클래스 구조

```
class EarlyStopping():
    def __init__(self, patience=5, verbose=False, delta=0, path='checkpoint.pt'):
        self.patience     = patience    # ① 개선 없을 때 기다릴 에포크 수
        self.verbose      = verbose
        self.delta        = delta       # ② 최소 개선량 (val_loss 변화가 delta 이하이면 “개선 없음
        self.path         = path        # 체크포인트 저장 경로

        self.counter      = 0           # 연속 개선 실패 횟수
        self.best_score   = None        # 지금까지 최저 검증 손실
        self.early_stop   = False       # True가 되면 학습 중단
        self.val_loss_min = np.Inf      # 초기 최저 손실을 무한대로 설정

    def __call__(self, val_loss, model):
        score = -val_loss  # score 높을수록 좋음(손실이 낮다는 의미)

        if self.best_score is None:
            # 최초 호출 시
            self.best_score = score
            self.save_checkpoint(val_loss, model)

        elif score < self.best_score + self.delta:
            # 개선이 없으면
            self.counter += 1
            if self.verbose:
                print(f'EarlyStopping counter: {self.counter} / {self.patience}')
            if self.counter >= self.patience:
                self.early_stop = True

        else:
            # 손실이 충분히 줄었으면
            self.best_score = score
            self.save_checkpoint(val_loss, model)
            self.counter = 0

    def save_checkpoint(self, val_loss, model):
        # 검증 손실이 감소할 때마다 모델 상태 저장
        if self.verbose:
            print(f'Validation loss improved ({self.val_loss_min:.6f} → {val_loss:.6f}).
        torch.save(model.state_dict(), self.path)
        self.val_loss_min = val_loss
```

- patience : 개선이 없는 에포크 수
- delta : 개선되었다고 간주할 최소 손실 감소량
- early_stop : True가 되면 학습 루프에서 중단

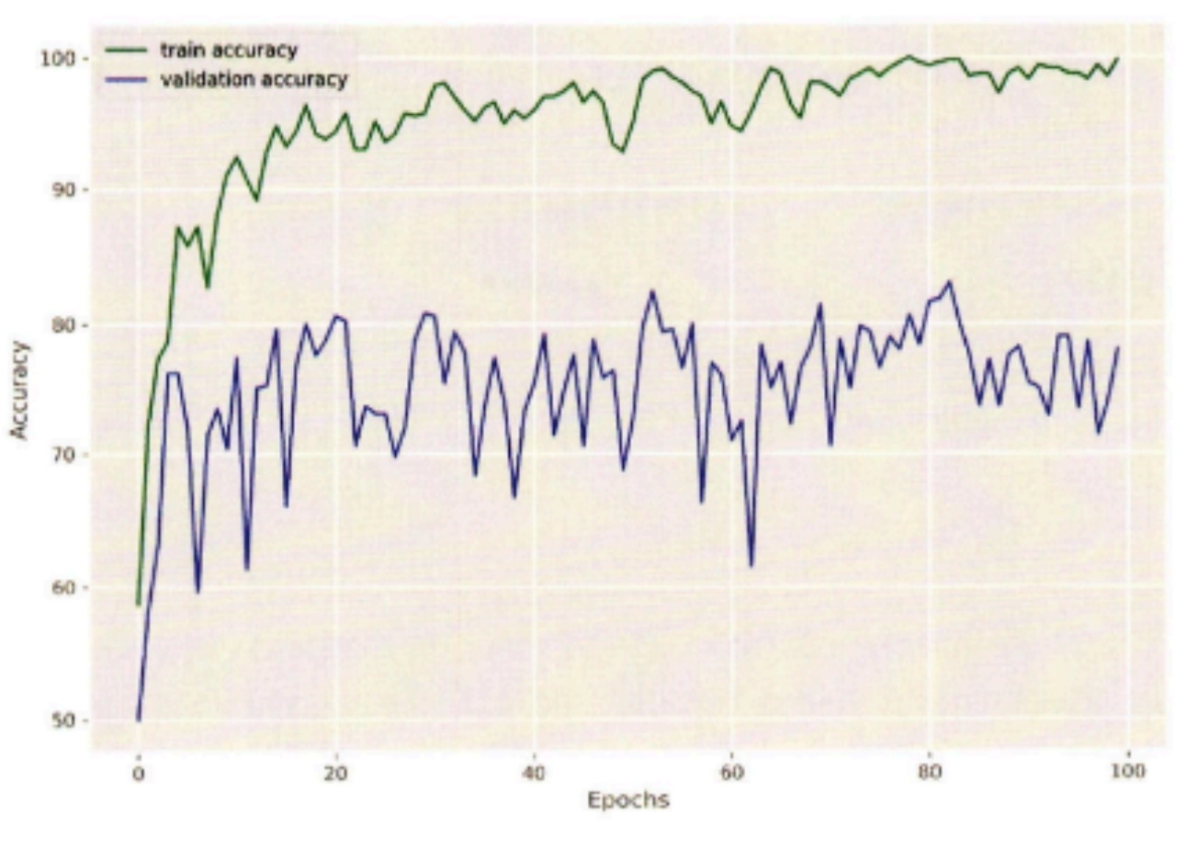
케라스(Keras) 콜백과 비교

```
from keras.callbacks import ModelCheckpoint, EarlyStopping

# 1) 모델 저장
ckpt = ModelCheckpoint('cp-{epoch:02d}.h5',
                       monitor='val_loss',
                       save_best_only=True,
                       verbose=1)

# 2) 조기 종료
es = EarlyStopping(monitor='val_loss',
                   patience=10,
                   verbose=1)

model.fit(..., callbacks=[ckpt, es])
```

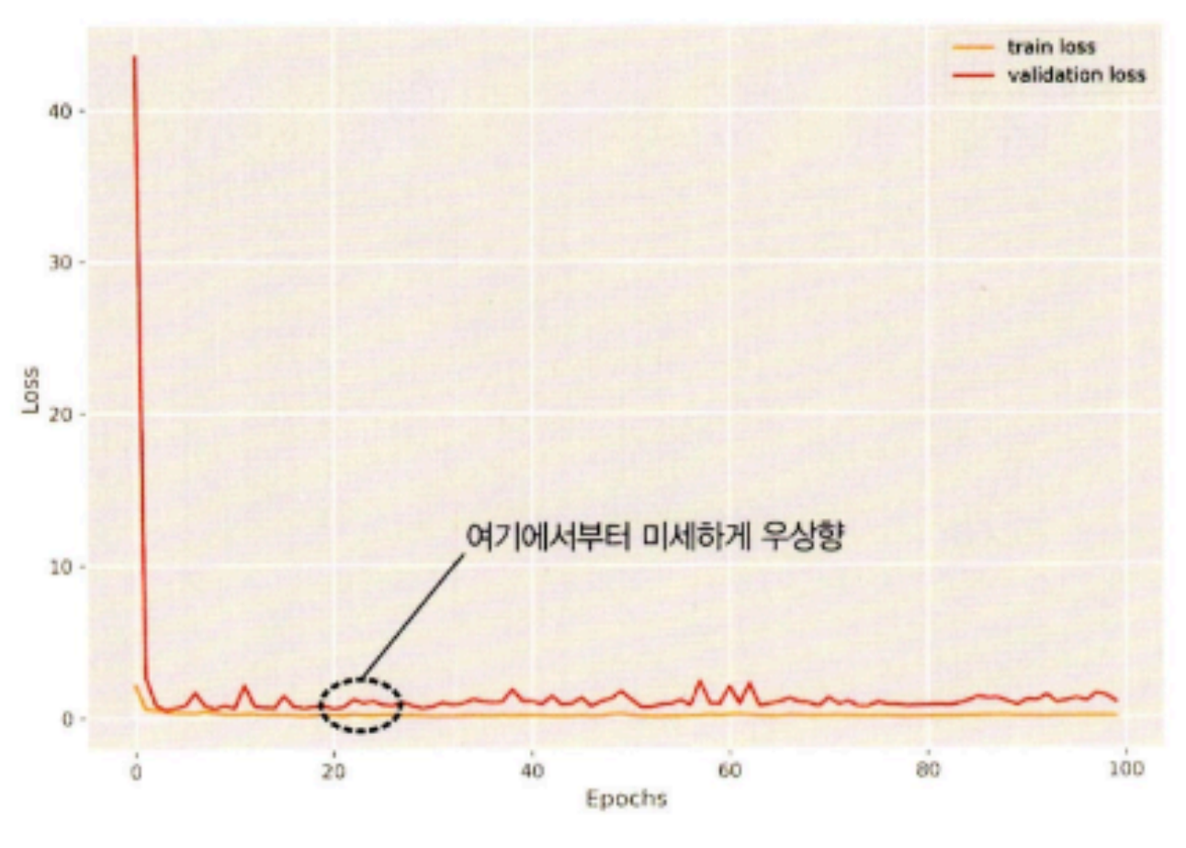
Keras는 미리 구현된 콜백 제공
PyTorch에서는 직접 EarlyStopping 클래스를 만들어 학습 루프에 통합해야 함

옵션별 학습 결과 변화 비교

어떤 인수도 적용하지 않았을 때



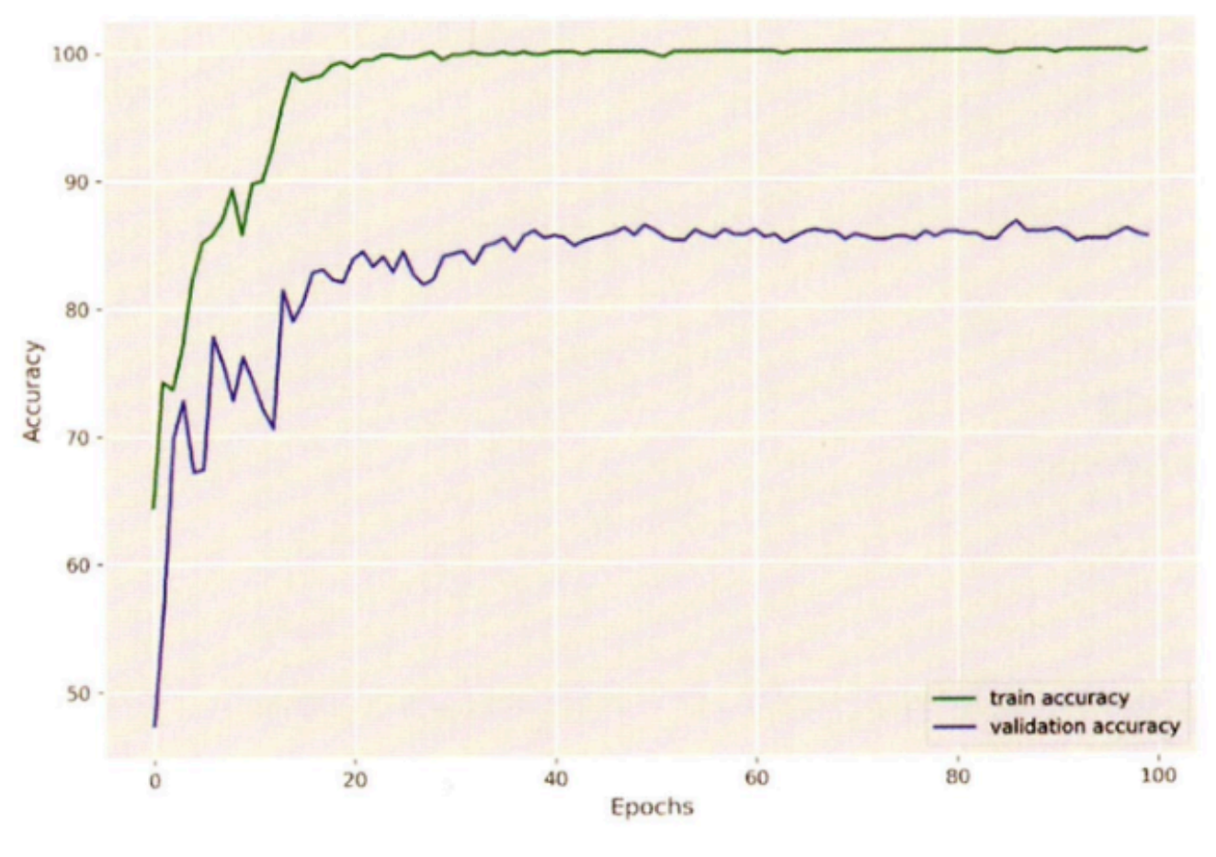
▼ 그림 8-55 어떤 인수도 적용되지 않았을 때의 오차



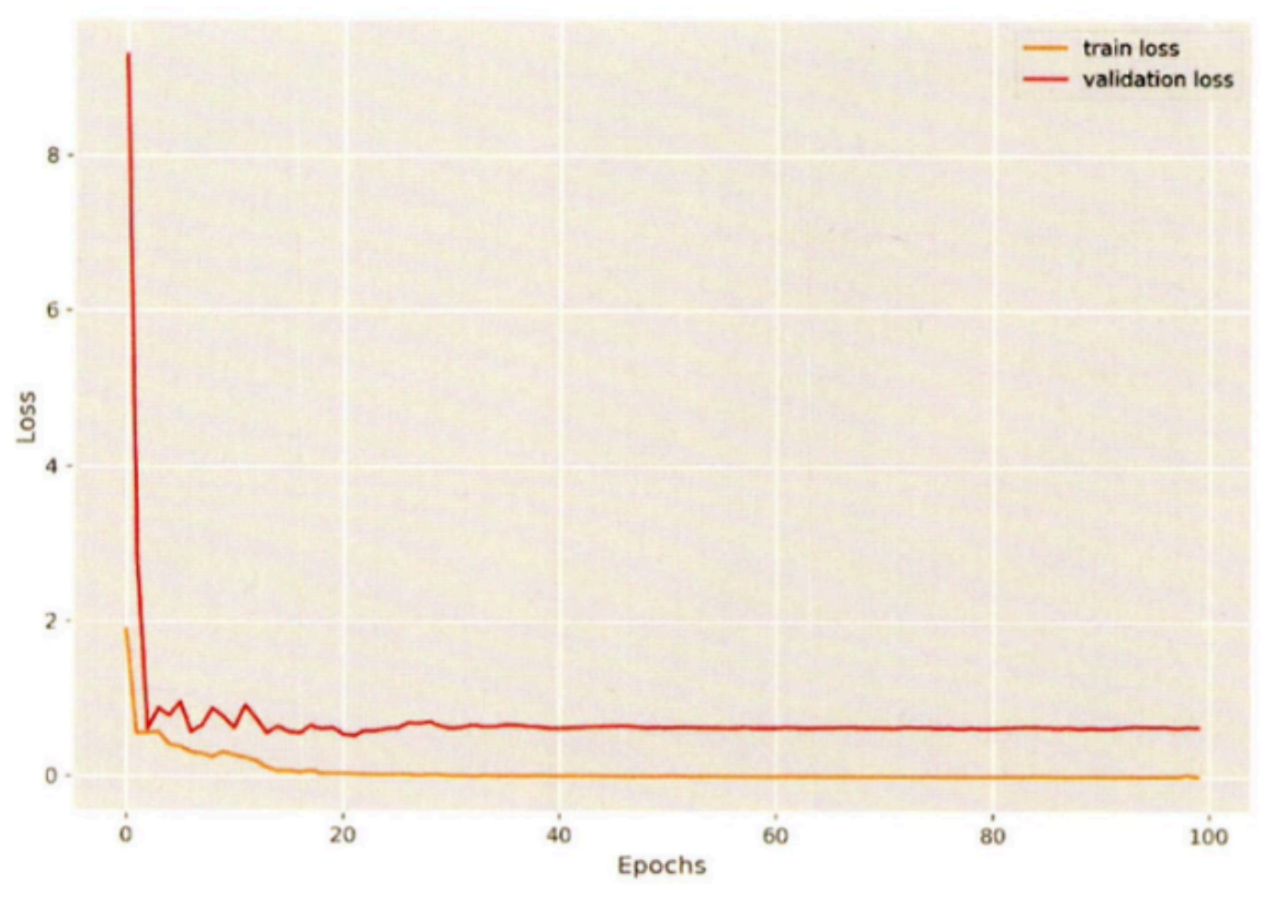
- 모델이 과적합(overfitting) 징후: 검증 성능이 변동이 심함
  → 학습률 값을 줄여야 함

--lr-scheduler 적용했을 때

▼ 그림 8-56 학습률 감소에 대한 인수가 적용되었을 때의 정확도



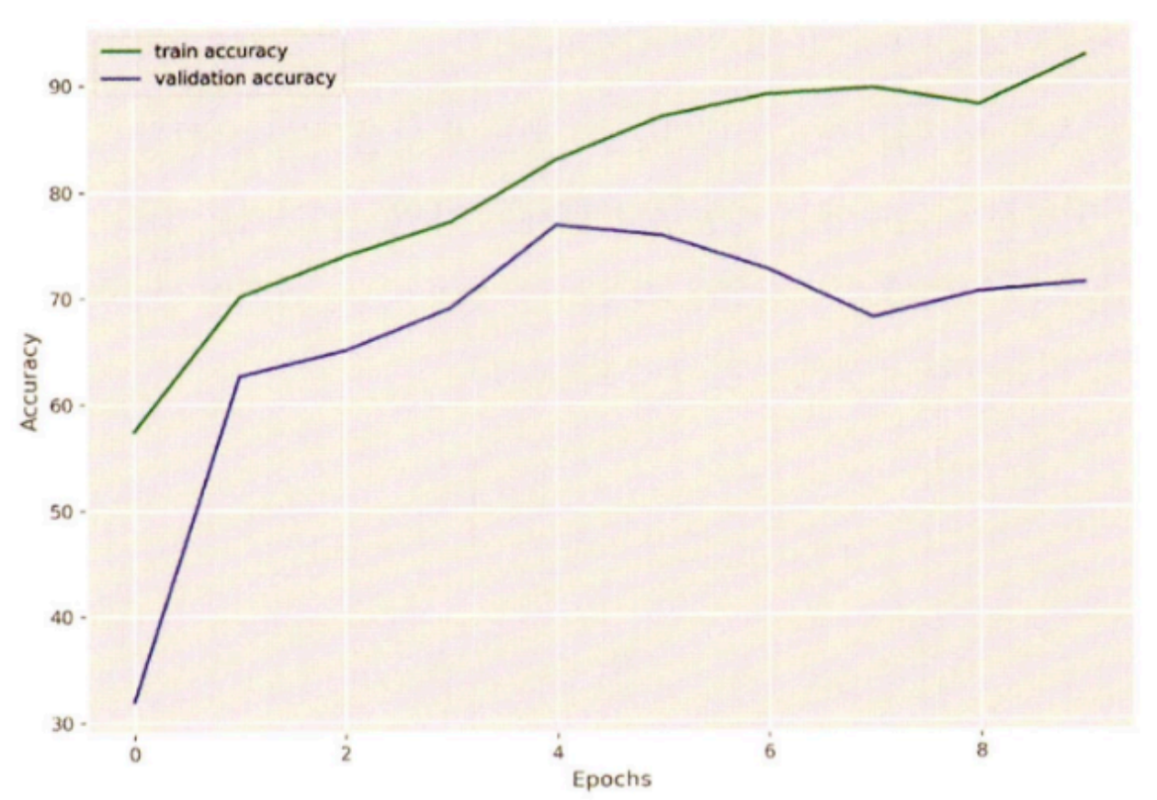
▼ 그림 8-57 학습률 감소에 대한 인수가 적용되었을 때의 오차



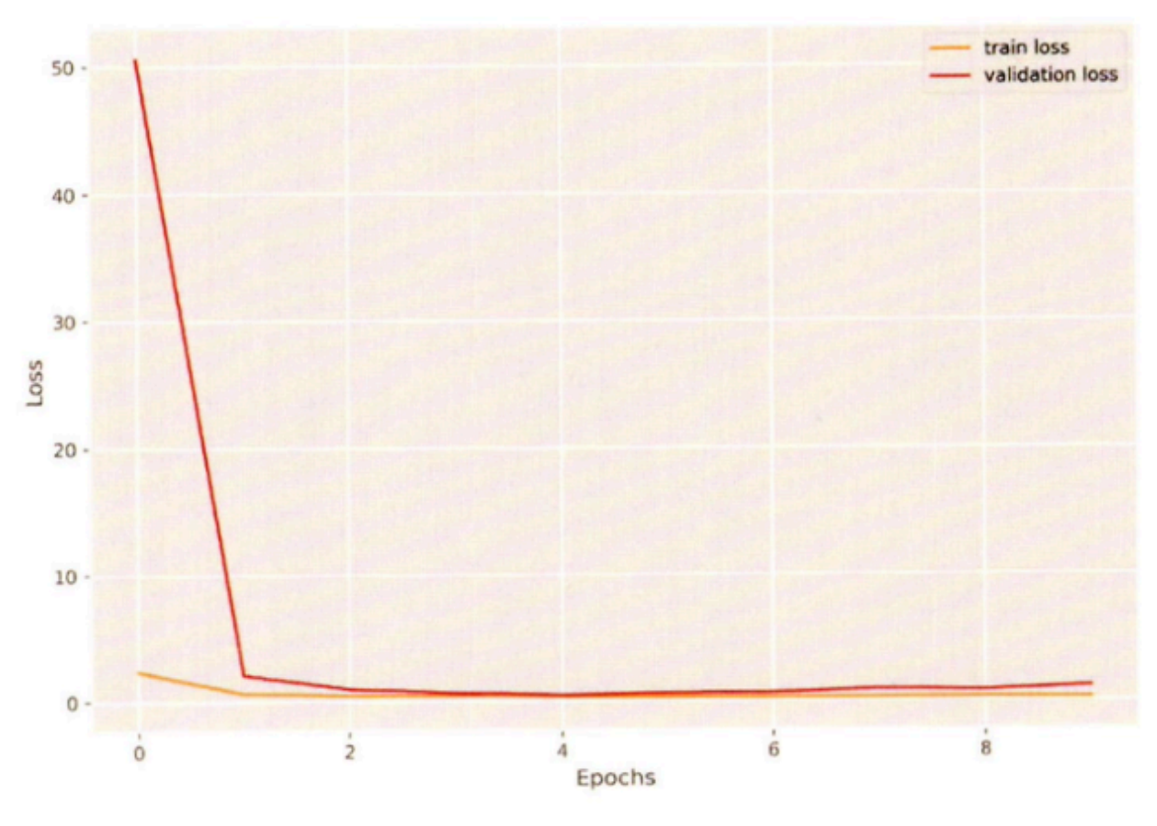
- 적절히 LR을 줄여서 후반부 학습 안정화 → 검증 성능 상승에 기여

--early-stopping 적용했을 때

▼ 그림 8-58 조기 종료에 대한 인수가 적용되었을 때의 정확도



▼ 그림 8-59 조기 종료에 대한 인수가 적용되었을 때의 오차



- 일정 에포크(patience) 동안 개선이 없으면 학습 조기 종료
- 불필요하게 과도한 학습을 막고 적절한 시점에 멈춰 과적합 최소화

chelseey

이전 포스트
Recurrent Neural Network

다음 포스트
12. Local Area Networks

0개의 댓글

댓글을 작성하세요

댓글 작성

관심 있을 만한 포스트

Powered by
Stellate